

PROGRAMACIÓN AVANZADA

Interfaz de Usuario (GUI)
Swing – Layout Manager



Ing. Matias Macrino

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

JOptionPane:

-> Este componente también es conocido como dialog boxes, el cual representa una ventana de dialogo para interfacear con el usuario.

-> Tiene tres métodos, mas comunes y utilizados:

- ❖ *JOptionPane.showInputDialog*
- ❖ *JOptionPane.showMessageDialog*
- ❖ *JOptionPane.showConfirmDialog*

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

JOptionPane (JOptionPane.showMessageDialog):

-> Generalmente se utiliza para mostrar un cuadro de dialogo al usuario. Su método por lo mínimo recibe dos argumentos.

Argumentos:

Component: el componente padre al que pertenece, puede ser **null**.

Object mensaje: es el objeto que corresponde al mensaje (texto) a mostrarse

String: texto que será el título del cuadro de diálogo.

int: que es el tipo de mensaje definido por una constante de la clase
JOptionPane

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

JOptionPane (JOptionPane.showInputDialog):

-> Es un mensaje de dialogo, que le permite al usuario ingresar información, que se es requerida.

Los parámetros que recibe son los mismo que **MessageDialog**

-> Una vez obtenido el dato que ingreso el usuario, se debe realizar una conversión al tipo de dato primitivo Java a utilizar. Ya que solamente retorna un tipo de dato String.

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

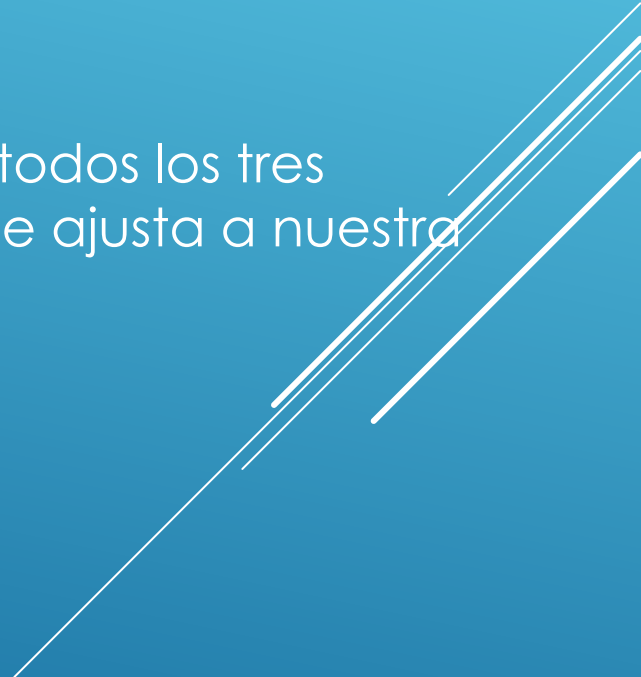
JOptionPane (JOptionPane.showConfirmDialog):

- > Este método generalmente sirve para solicitar al usuario la confirmación de una determinado acción ocurrida o a ocurrir.
- > Los parámetros a utilizar son los mismos que **MessageDialog**, con la diferencia que las constantes del tipo de mensaje son las siguientes: **YES_NO_OPTION**, **YES_NO_CANCEL_OPTION**, **YES_OPTION**, **NO_OPTION**, **CANCEL_OPTION**
- > El resultado que retorna es un entero que corresponde a la opción seleccionado por el usuario.

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

JOptionPane (JOptionPane.showOptionDialog):

-> Este método no mencionado anteriormente, es el mas distintos de todos los tres anteriores, ya que podemos decir que es un cuadro de dialogo que se ajusta a nuestra necesidad, gracias a que es totalmente configurable.

Several white diagonal lines of varying lengths and thicknesses are positioned in the bottom right corner of the slide, creating a modern, abstract graphic element.

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

JOptionPane (JOptionPane. showOptionDialog):

-> Parametros que recibe:

- *Component:* el objeto que indica de qué componente es hijo.
- *Objet:* mensaje que es un String que corresponde al texto a mostrarse como mensaje.
- *Titulo:* String que se establecerá como titulo de la ventana.
- *Tipo de opción:* es un entero, representado por unas constantes que definen qué opciones tendrá el cuadro de diálogo.
- *TipoDeMensaje:* entero que determina el tipo de mensaje (ERROR_MESSAGE, por ejemplo)
- *Icono:* la imagen que acompañará al mensaje, si no se especifica (es decir, se pasa null) se establecerá uno por defecto de acuerdo al tipoDeMensaje.
- *Opciones:* un array tipo Object que indica las opciones posibles
- *ValorInicial:* es la opción predeterminada, deberá ser una de las opciones introducidas en el array de opciones. Puede ser null.

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

JOptionPane (JOptionPane.showOptionDialog):

-> Retorna un entero, el cual corresponde a la opción seleccionada por el usuario.

Ejemplo:

```
int opcion = JOptionPane.showOptionDialog(null, "Este es un cuadro de dialogo Option",  
"showOptionDialog", JOptionPane.YES_NO_CANCEL_OPTION,  
JOptionPane.QUESTION_MESSAGE, null, new Object[] {"Si", "No", "Cancelar"}, "Si");
```



APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

JOptionPane:

<u>Message Dialog Type</u>	<u>Icon</u>	<u>Description</u>
ERROR_MESSAGE		Indica error del usuario
INFORMATION_MESSAGE		Mensaje informativo
WARNING_MESSAGE		Aviso: potencial problema
QUESTION_MESSAGE		Pregunta al usuario
PLAIN_MESSAGE	sin icono	Mensaje sin icono

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

JOptionPane:

-> Ejemplo:

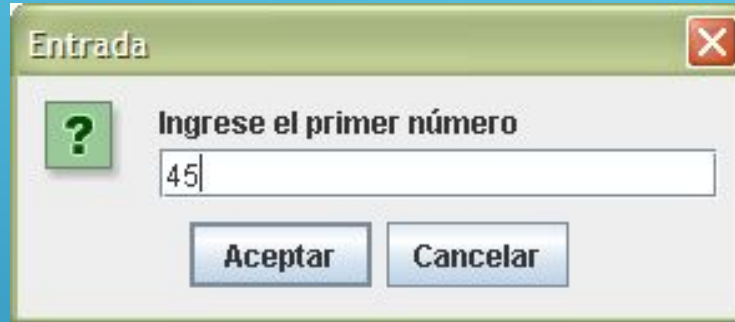
```
public static void main(String[] args) {  
    String sPrimerNro = JOptionPane.showInputDialog("Ingrese el primer número");  
    String sSegundoNro = JOptionPane.showInputDialog("Ingrese el segundo  
        número");  
  
    int PrimerNro = Integer.parseInt(sPrimerNro);  
    int SegundoNro = Integer.parseInt(sSegundoNro);  
    int suma = PrimerNro + SegundoNro;  
  
    JOptionPane.showMessageDialog(null, "El resultado es: " + suma, "Suma de  
        Enteros", JOptionPane.PLAIN_MESSAGE);  
}
```

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

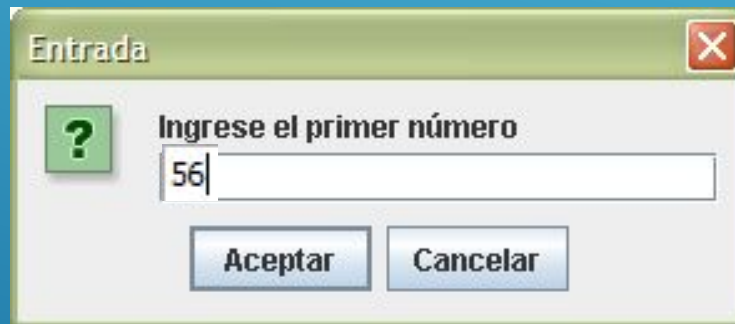
JOptionPane:

-> Ejemplo:

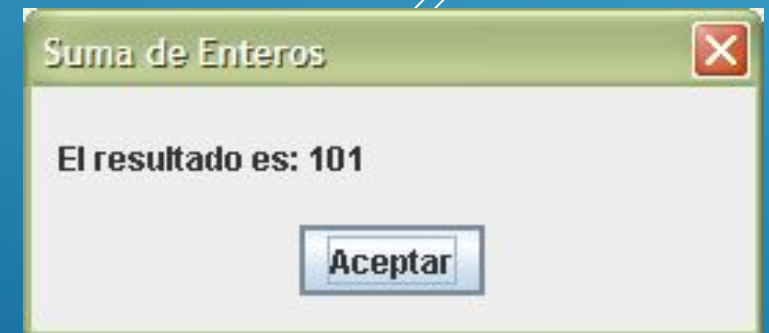
```
JOptionPane.showInputDialog("Ingrese el primer número");
```



```
JOptionPane.showInputDialog("Ingrese el segundo número");
```



```
JOptionPane.showMessageDialog(null, "El resultado es: " + suma,  
    "Suma de Enteros", JOptionPane.PLAIN_MESSAGE);
```



APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Menús:

Los menús permiten al usuario ejecutar diferentes tipo de acciones. Estos son una parte de GUI.

Para generar el menú se utilizan las siguientes clases "**JMenuBar**", "**JMenu**", "**JMenuItem**"

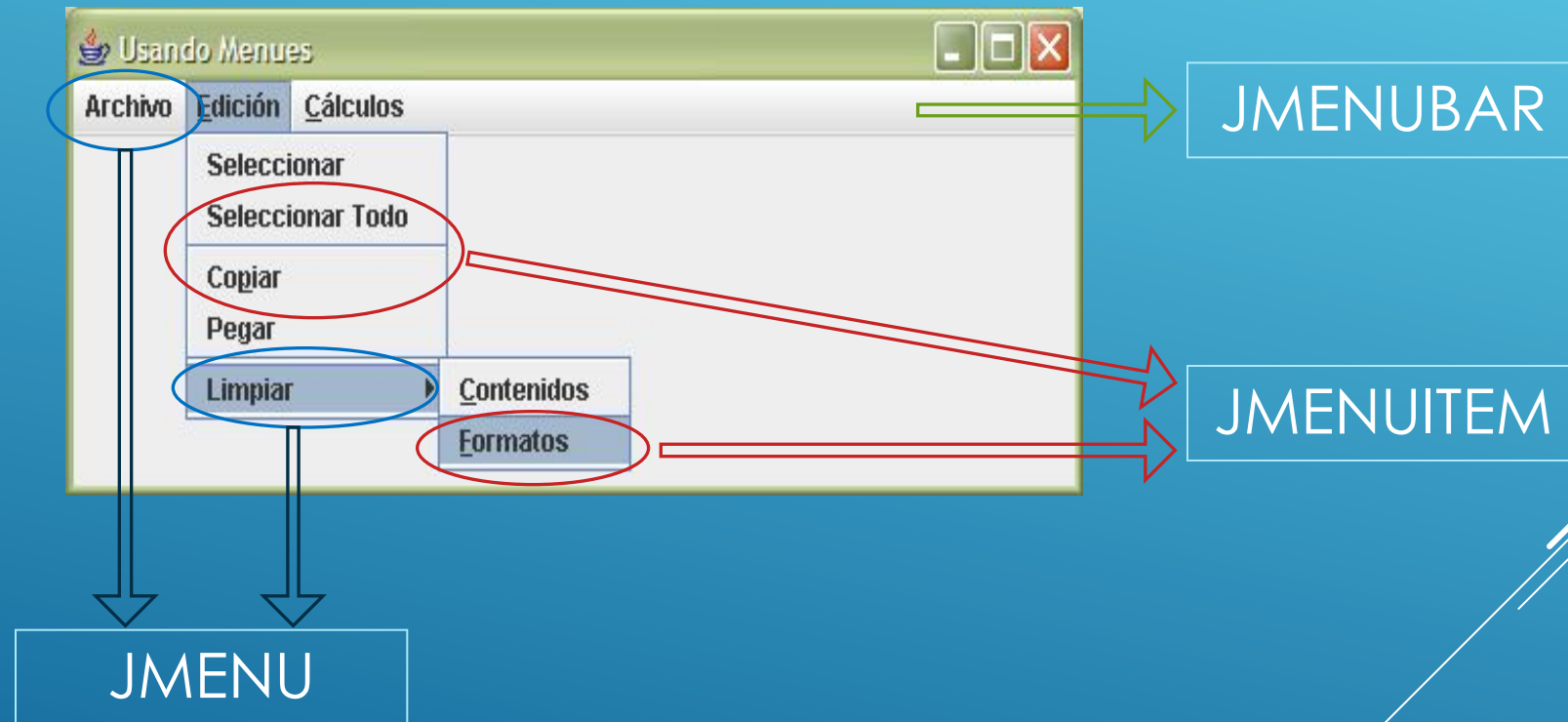
JMenuBar: contiene los métodos necesarios para el manejo de la barra del menú.

JMenu: contiene los métodos necesarios para todo el manejo del menú. Los menús contienen ítems que son agregados a la barra de menú como también a submenús. Cuando un menú es seleccionado, se expande para mostrar la lista de sus ítems.

JMenuItem: contiene los métodos necesarios para el manejo de los ítems del menú. Un ítem de menú es un componente que al ser seleccionado genera un evento.

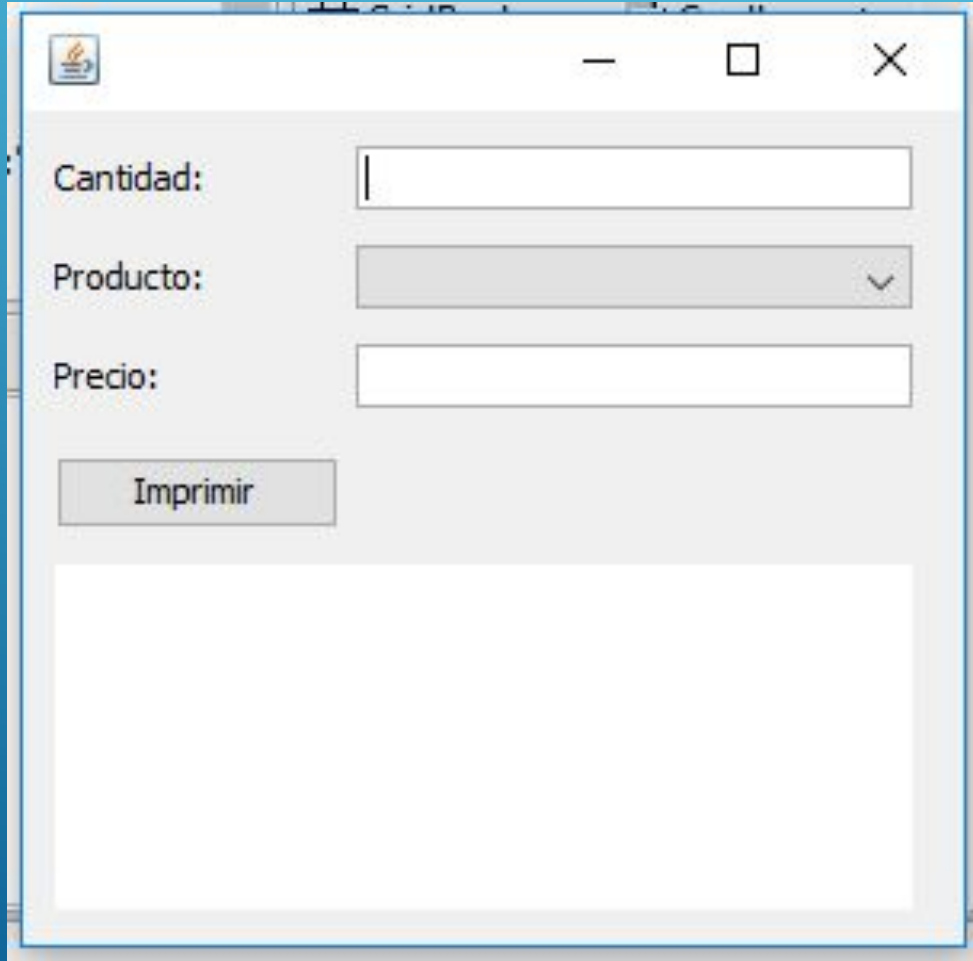
APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Menús:



APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

JOptionPane - Practica:

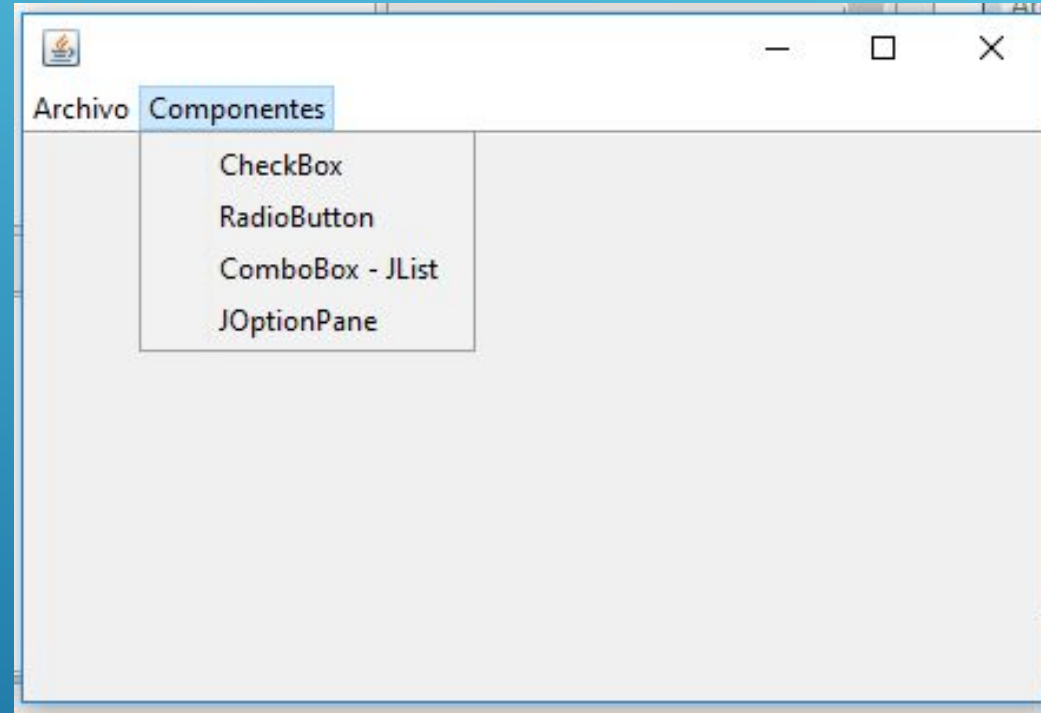


A screenshot of a Java Swing window titled "Formulario de Datos". The window has a standard title bar with a minimize button, a maximize button, and a close button. Inside the window, there are three input fields: "Cantidad:" with a text box, "Producto:" with a dropdown menu, and "Precio:" with a text box. Below these fields is a button labeled "Imprimir". At the bottom of the window is a large, empty rectangular area, likely for a receipt or additional information.

-Validar que todos los campos se encuentren cargados, en caso de encontrarse alguno incompleto se tiene que visualizar un ***JOptionPane*** informando que campo no se encuentra completo.

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Menús - Practica:



- Cada una de las opciones deberá de abrir la venta de practica ya creada.
- Menú "Archivo", agregar opción salir para cerrar la aplicación.

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Gestores de organización “Layout Manager”

Los layout son objetos que proporcionan una colocación y tamaño automáticos de los componentes dentro de un contenedor, siguiendo los criterios definidos por la clase a la que pertenece.

Estos establecen una disposición relativa de los componentes dentro del contenedor, si se modifica el tamaño de este se reconfiguran las dimensiones de los componentes.

- ✓ FlowLayout
- ✓ BorderLayout
- ✓ GridLayout
- ✓ CardLayout
- ✓ AbsoluteLayout

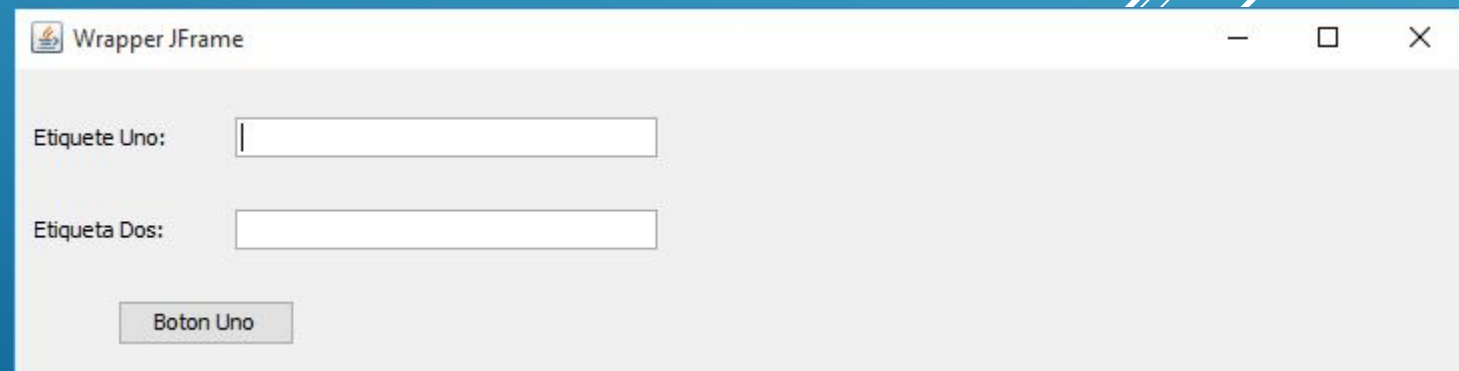
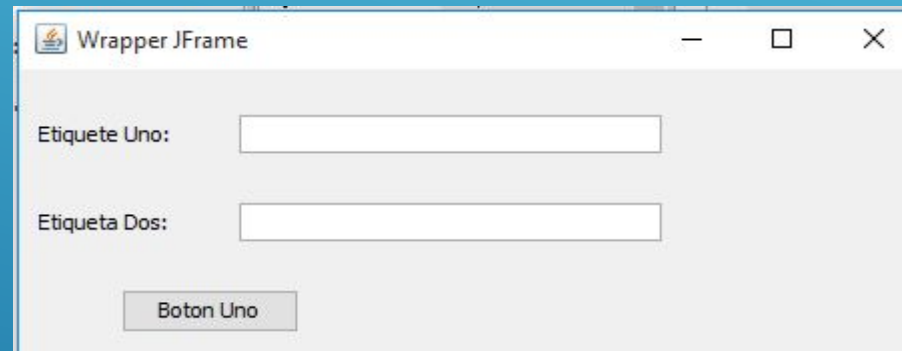
APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

ABSOLUTE LAYOUT

En este tipo de **layout** los componentes se pueden colocar en cualquier parte del contendor. Con lo que hay que tener en cuenta es que a cada componente que se agregue se le debe definir la posición y tamaño del mismo.

Cuenta con la desventaja frente al resto que al reajustar el contendor, los componentes no se reajustaran con el mismo.

```
public class PanelAbsoluteLayout extends JPanel {  
  
    public PanelAbsoluteLayout() {  
        this.setLayout(null);  
  
        JLabel lblUno = new JLabel("Etiquete Uno:");  
        lblUno.setBounds(10, 27, 83, 14);  
        JTextField textFieldUno = new JTextField();  
        textFieldUno.setBounds(110, 24, 211, 20);  
        JLabel lblDos = new JLabel("Etiqueta Dos:");  
        lblDos.setBounds(10, 73, 83, 14);  
        JTextField textFieldDos = new JTextField();  
        textFieldDos.setBounds(110, 70, 211, 20);  
        JButton btnUno = new JButton("Boton Uno");  
        btnUno.setBounds(51, 115, 89, 23);  
  
        this.add(lblUno);  
        this.add(textFieldUno);  
        this.add(lblDos);  
        this.add(textFieldDos);  
        this.add(btnUno);  
    }  
}
```

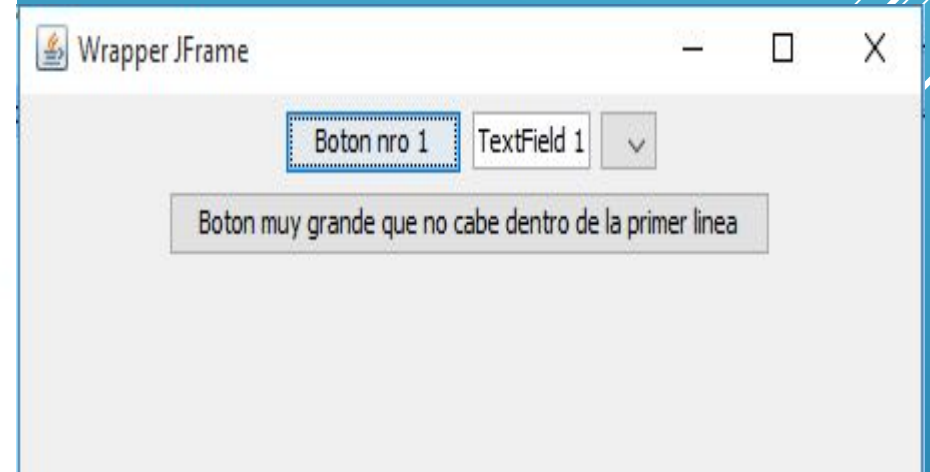


APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

FLOW LAYOUT

Este tipo de layout coloca los componentes siguiendo un orden de izquierda a derecha y de arriba hacia abajo. Por defecto, cada línea de componentes se encuentra centrada en el contenedor, pudiendo definir una alineación diferente en la creación del gestor.

```
public class PanelFlowLayout extends JPanel {  
  
    public PanelFlowLayout() {  
        this.setLayout(new FlowLayout(FlowLayout.CENTER, 5, 5));  
  
        JButton btnUno = new JButton("Boton nro 1");  
        JTextField txtTextfield = new JTextField();  
        txtTextfield.setText("TextField 1");  
        JComboBox comboBox = new JComboBox();  
        JButton btnNewButton = new JButton("Boton muy grande que no cabe dentro de la primer linea");  
  
        this.add(btnUno);  
        this.add(txtTextfield);  
        this.add(comboBox);  
        this.add(btnNewButton);  
  
    }  
}
```



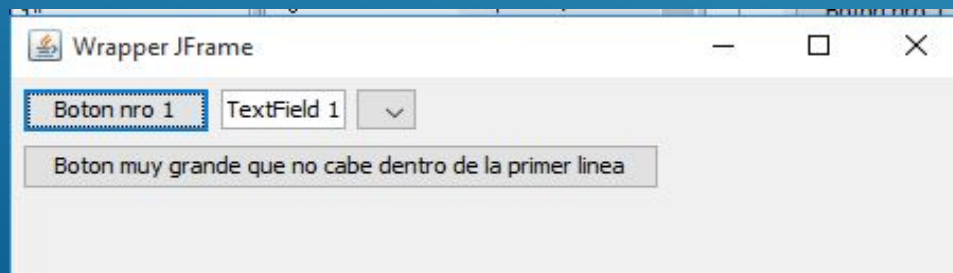
APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

FLOW LAYOUT

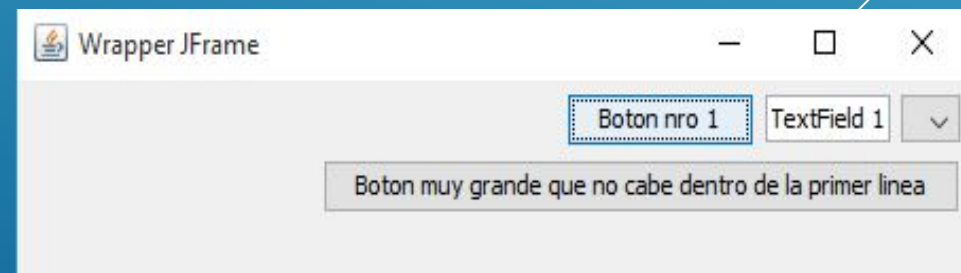
Alineación del Layout: Los valores mas comunes que existen para la alineación de este gestor son:

- Center: Alineación centrada.
- LEFT: Alineación izquierda.
- RIGHT: Alineación derecha.

```
public PanelFlowLayout() {  
    this.setLayout(new FlowLayout(FlowLayout.LEFT, 5, 5));  
  
    JButton btnUno = new JButton("Boton nro 1");  
    JTextField txtTextfield = new JTextField();  
}
```



```
public PanelFlowLayout() {  
    this.setLayout(new FlowLayout(FlowLayout.RIGHT, 5, 5));  
  
    JButton btnUno = new JButton("Boton nro 1");  
    JTextField txtTextfield = new JTextField();  
}
```



APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

BORDER LAYOUT

Este tipo de layout divide el contenedor en cinco regiones: norte, sur, este, oeste y centro. Para añadir componentes en un contenedor de este tipo, existe una versión sobrecargada del método `add()`.

```
add(Component cmp, Object constraints)
```

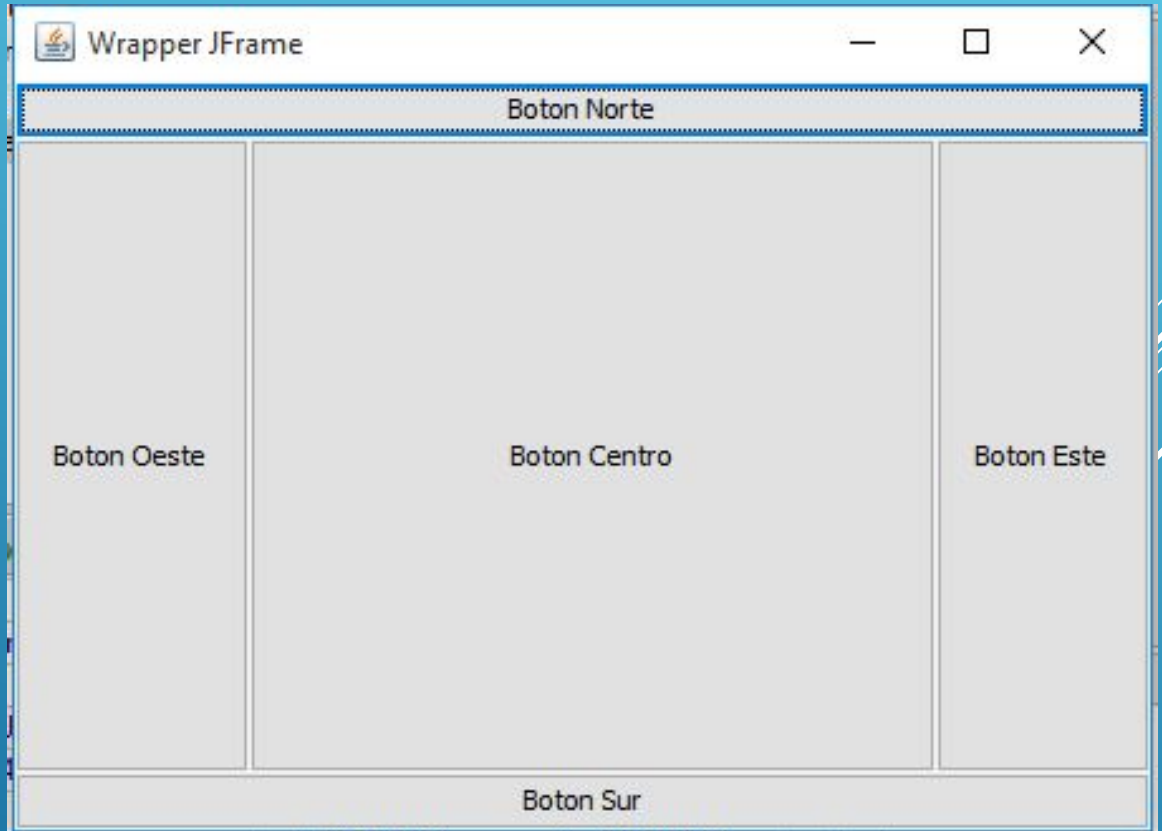
Los valores que puede tomar el argumento `constraints` son la definidas en las siguientes constantes del tipo `String`:

- `NORTH`
- `SOUTH`
- `EAST`
- `WEST`
- `CENTER`

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

BORDER LAYOUT

```
public class PanelBorderLayout extends JPanel {  
  
    public PanelBorderLayout() {  
        this.setLayout(new BorderLayout(0, 0));  
  
        JButton btnNorth = new JButton("Boton Norte");  
        JButton btnWest = new JButton("Boton Oeste");  
        JButton btnSouth = new JButton("Boton Sur");  
        JButton btnEast = new JButton("Boton Este");  
        JButton btnCenter = new JButton("Boton Centro");  
  
        this.add(btnNorth, BorderLayout.NORTH);  
        this.add(btnWest, BorderLayout.WEST);  
        this.add(btnSouth, BorderLayout.SOUTH);  
        this.add(btnEast, BorderLayout.EAST);  
        this.add(btnCenter, BorderLayout.CENTER);  
  
    }  
}
```

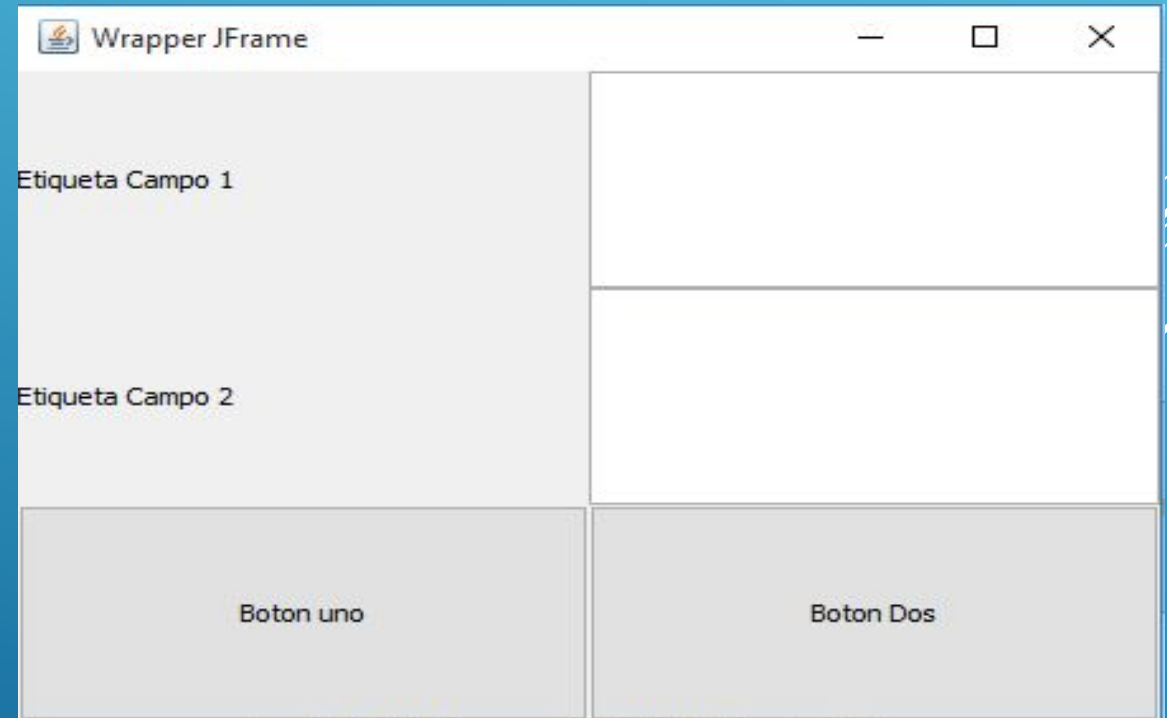


APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

GRID LAYOUT

Este tipo de layout divide el contenedor en filas y columnas, formando una especie de rejilla de celdas rectangulares con igual tamaño. Cada una esta ocupada por un solo componente.

```
public class PanelGridLayout extends JPanel {  
  
    public PanelGridLayout() {  
        this.setLayout(new GridLayout(3, 2, 0, 0));  
  
        JLabel lblUno = new JLabel("Etiqueta Campo 1");  
        JTextField textFieldUno = new JTextField();  
        JLabel lblDos = new JLabel("Etiqueta Campo 2");  
        JTextField textFieldDos = new JTextField();  
        JButton btnUno = new JButton("Boton uno");  
        JButton btnDos = new JButton("Boton Dos");  
  
        this.add(lblUno);  
        this.add(textFieldUno);  
        this.add(lblDos);  
        this.add(textFieldDos);  
        this.add(btnUno);  
        this.add(btnDos);  
    }  
}
```

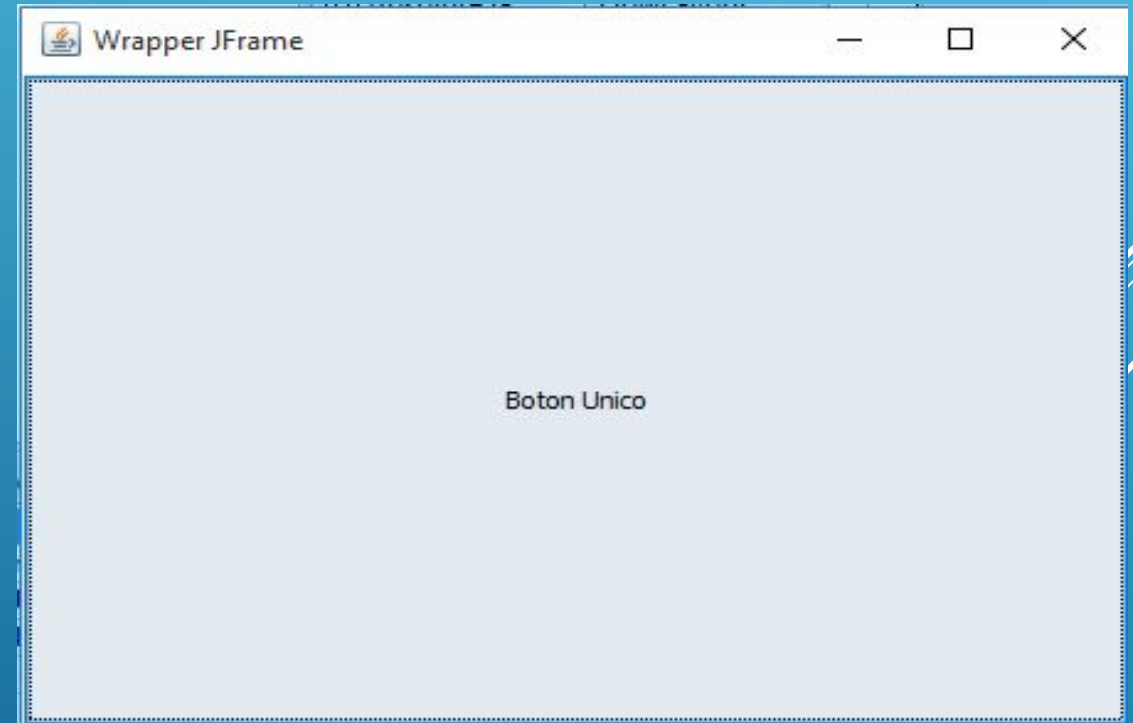


APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

CARD LAYOUT

En este tipo de layout cada componente es apilado encima del anterior ocupando completamente todo el área del contenedor, de modo que solamente es visible un componente en cada momento.

```
public class PanelCardLayout extends JPanel {  
  
    public PanelCardLayout() {  
        this.setLayout(new CardLayout(0, 0));  
        JButton btnUnico = new JButton("Boton Unico");  
        this.add(btnUnico);  
    }  
}
```



APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

*Interface **KeyListener**:*

Dicha interface nos provee todos los métodos necesarios para poder controlar los eventos del teclado.

Al implementar la interface, nos obliga a sobrescribir los siguientes tres métodos:

KeyPressed(KeyEvent e)

KeyReleased(KeyEvent e)

KeyTyped(KeyEvent e)

Los tres métodos reciben una instancia **KeyEvent** que nos permite conocer que evento sucedió desde el teclado.

KeyPressed: este método se ejecuta cuando el usuario presiona una tecla.

KeyReleased: este método se ejecuta cuando el usuario libera una tecla.

KeyTyped: este método se ejecuta cuando el usuario presiona una tecla, pero solo cuando corresponde a caracteres, teclas especiales como F1, F2, etc.

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

*Interface **KeyListener**:*

¿Qué nos proporciona **KeyEvent**?

- ✓ **KeyEvent** tiene constantes para saber que tecla fue pulsada a las mismas puedes obtenerlas a través de ***KeyEvent.VK_tecla***
- ✓ Otra de las maneras de saber que tecla fue pulsada es a través del método ***getKeyCode()*** que nos retorna un entero con el valor de la constante a la tecla que fue pulsada.
- ✓ A través del método ***getKeyText(int KeyCode)*** obtendremos en texto la tecla que fue pulsada.

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

*Interface **MouseListener**:*

Dicha interface nos provee todos los métodos necesarios para poder controlar los eventos del mouse.

Al implementar la interface, nos obliga a sobrescribir los siguientes tres métodos:

MouseReleased(MouseEvent e)

MousePressed(MouseEvent e)

MouseExited(MouseEvent e)

MouseEntered(MouseEvent e)

MouseClicked(MouseEvent e)

Los cinco métodos reciben una instancia **MouseEvent** que nos permite conocer que evento sucedió desde el mouse

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Interface MouseListener:

- ▣ **MouseReleased(MouseEvent e):** Es utilizado para saber si el mouse soltó el objeto
- ▣ **MousePressed(MouseEvent e):** Es utilizada para saber si el mouse presiono el objeto.
- ▣ **MouseExited(MouseEvent e):** Es utilizado para saber si el puntero del mouse salio del área del objeto
- ▣ **MouseEntered(MouseEvent e):** Es utilizada para saber si el puntero del mouse ingreso al área del objeto.
- ▣ **MouseClicked(MouseEvent e):** Es utilizado para saber si se presiono el click, es decir si presionan o sueltan el botón del mouse.
- ▣ El método que mas llegaremos a utilizar es el **getClickCount()** que nos retorna la cantidad de click realizados sobre el objeto.

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Eventos asociados a componentes:

- Lista de eventos más comunes, junto a la interface que debe implementar el objeto que escuche esos eventos y el método para asociar un objeto para escuchar dichos eventos

Event, listener interface y métodos para ligar el objeto que escucha	Componentes que generan este tipo de eventos
ActionEvent ActionListener addActionListener()	JButton, JList, JTextField JMenuItem JCheckBox, JMenuItem, JMenu, JPopupMenu

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Eventos asociados a componentes:

AdjustmentEvent AdjustmentListener addAdjustmentListener()	JScrollBar y cualquier componente que implemente la interfaz Adjustable
ComponentEvent ComponentListener addComponentListener()	Component, JButton, JCanvas, JCheckBox, JComboBox, Container, JPanel, JApplet, JFrame, JDialog, JLabel, JList, JScrollbar, JTextArea, JTextField
ContainerEvent ContainerListener addContainerListener()	Container, JPanel, JApplet, JScrollPane, JFrame

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Eventos asociados a componentes:

FocusEvent FocusListener addFocusListener()	Component, JButton, JCanvas, JCheckBox, JComboBox, Container, JPanel, JApplet, JFrame, JDialog, JLabel, JList, JScrollbar, JTextArea, JTextField
KeyEvent KeyListener addKeyListener()	Component, JButton, JCanvas, JCheckBox, JComboBox, Container, JPanel, JApplet, JFrame, JDialog, JLabel, JList, JScrollbar, JTextArea, JTextField
MouseEvent MouseListener addMouseListener()	Component, JButton, JCanvas, JCheckBox, JComboBox, Container, JPanel, JApplet, JFrame, JDialog, JLabel, JList, JScrollbar, JTextArea, JTextField

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Eventos asociados a componentes:

FocusEvent FocusListener addFocusListener()	Component, JButton, JCanvas, JCheckBox, JComboBox, Container, JPanel, JApplet, JFrame, JDialog, JLabel, JList, JScrollbar, JTextArea, JTextField
KeyEvent KeyListener addKeyListener()	Component, JButton, JCanvas, JCheckBox, JComboBox, Container, JPanel, JApplet, JFrame, JDialog, JLabel, JList, JScrollbar, JTextArea, JTextField
MouseEvent MouseListener addMouseListener()	Component, JButton, JCanvas, JCheckBox, JComboBox, Container, JPanel, JApplet, JFrame, JDialog, JLabel, JList, JScrollbar, JTextArea, JTextField

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Eventos asociados a componentes:

TextEvent TextListener addTextListener()	JTextComponent, JTextArea, JTextField
ListSelectionEvent ListSelectionListener addListSelectionListener()	JList

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Eventos asociados a componentes:

- Principales interfaces junto con sus clases "Adapter" y los métodos que poseen

Listener Interface y Adapter	Métodos Ejecutores
ActionListener	actionPerformed(ActionEvent)
AdjustmentListener	adjustmentValueChanged(AdjustmentEvent)
ComponentListener ComponentAdapter	componentHidden(ComponentEvent) componentShow(ComponentEvent) componentMoved(ComponentEvent) componentResized(ComponentEvent)

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Eventos asociados a componentes:

ContainerListener ContainerAdapter	componentAdded(ContainerEvent) componentRemoved(ContainerEvent)
FocusListener FocusAdapter	focusGained(FocusEvent) focusLost(FocusEvent)
KeyListener KeyAdapter	keyPressed(KeyEvent) keyReleased(KeyEvent) keyTyped(KeyEvent)
MouseListener MouseAdapter	mouseClicked(MouseEvent) mouseEntered(MouseEvent) mouseExited(MouseEvent)

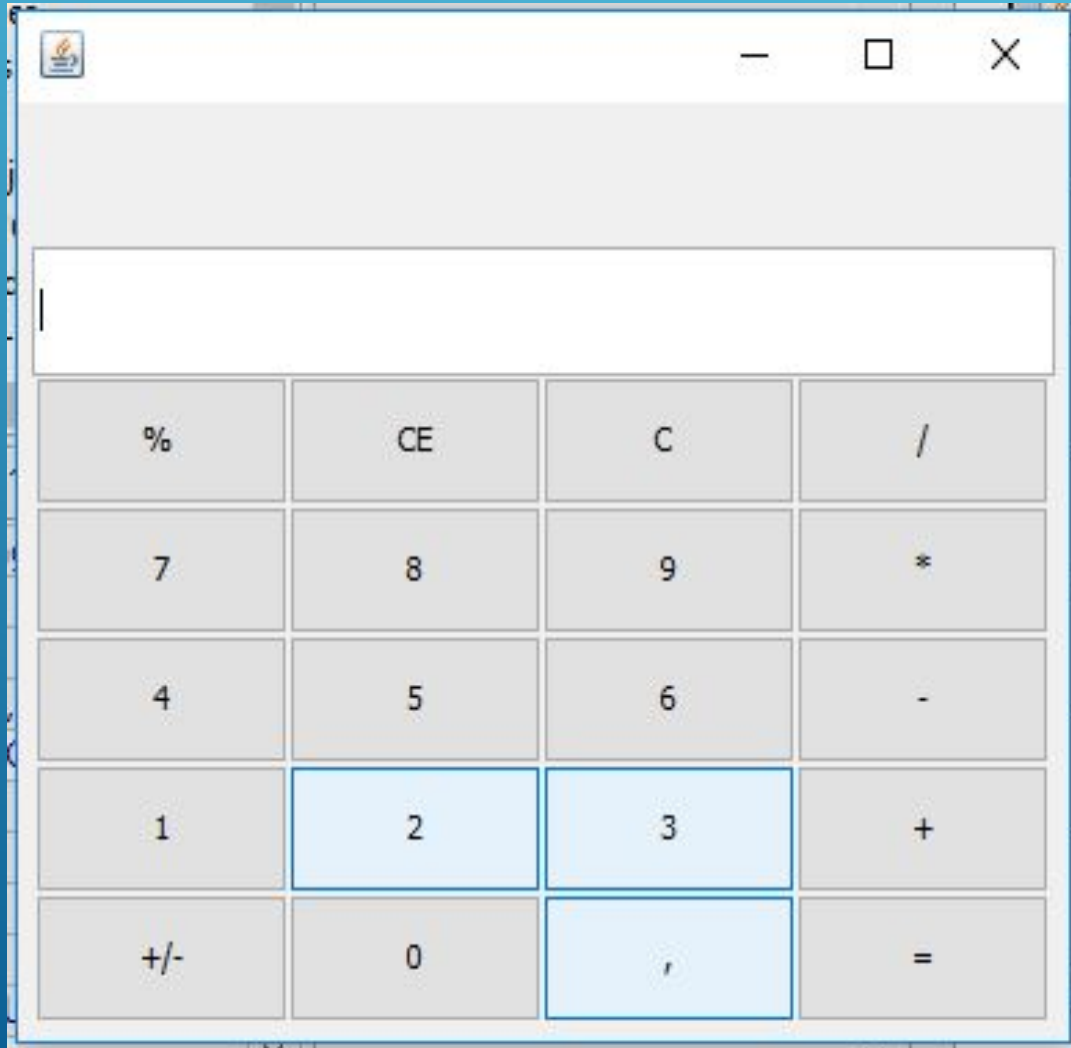
APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Eventos asociados a componentes:

MouseListener	mousePressed(MouseEvent)
MouseAdapter	mouseReleased(MouseEvent)
MouseMotionListener	mouseDragged(MouseEvent)
MouseMotionAdapter	mouseMoved(MouseEvent)
ItemListener	itemStateChanged(ItemEvent)
WindowListener	windowOpened(WindowEvent)
WindowAdapter	windowClosed(WindowEvent)
	windowActivated(WindowEvent)
	windowDeactivated(WindowEvent)
ListSelectionListener	valueChanged(ListSelectionEvent)

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Practica (Calculadora Avanzada)



- Utilizar Layout
- Utilizar Objetos
- Dividir eventos de pantalla
- En caso de errores utilizar cuadros de diálogos
- Utilizar KeyListener